

Работа с файловой системой

Класс Directory, классы File и FileInfo, FileStream.

Класс Directory

Для работы с каталогами в пространстве имен System.IO предназначены сразу два класса: **Directory** и **DirectoryInfo**.

Статический класс **Directory** предоставляет ряд методов для управления каталогами. Некоторые из этих методов:

- **CreateDirectory(path)**: создает каталог по указанному пути path
- **Delete(path)**: удаляет каталог по указанному пути path
- **Exists(path)**: определяет, существует ли каталог по указанному пути path. Если существует, возвращается true, если не существует, то false
- **GetCurrentDirectory()**: получает путь к текущей папке
- **GetDirectories(path)**: получает список подкаталогов в каталоге path
- **GetFiles(path)**: получает список файлов в каталоге path
- **GetFileSystemEntries(path)**: получает список подкаталогов и файлов в каталоге path
- **Move(sourceDirName, destDirName)**: перемещает каталог
- **GetParent(path)**: получение родительского каталога
- **GetLastWriteTime(path)**: возвращает время последнего изменения каталога
- **GetLastAccessTime(path)**: возвращает время последнего обращения к каталогу
- **GetCreationTime(path)**: возвращает время создания каталога

Класс DirectoryInfo

Данный класс предоставляет функциональность для создания, удаления, перемещения и других операций с каталогами. Во многом он похож на Directory, но не является статическим.

Для создания объекта класса DirectoryInfo применяется конструктор, который в качестве параметра принимает путь к каталогу: *public DirectoryInfo (string path);*

Основные методы класса DirectoryInfo:

- **Create()**: создает каталог
- **CreateSubdirectory(path)**: создает подкаталог по указанному пути path
- **Delete()**: удаляет каталог
- **GetDirectories()**: получает список подкаталогов папки в виде массива DirectoryInfo
- **GetFiles()**: получает список файлов в папке в виде массива FileInfo
- **MoveTo(destDirName)**: перемещает каталог

Основные свойства класса DirectoryInfo:

- **CreationTime**: представляет время создания каталога
- **LastAccessTime**: представляет время последнего доступа к каталогу
- **LastWriteTime**: представляет время последнего изменения каталога
- **Exists**: определяет, существует ли каталог
- **Parent**: получение родительского каталога
- **Root**: получение корневого каталога
- **Name**: имя каталога
- **FullName**: полный путь к каталогу

Ссылка: 0

```
static void Main(string[] args)
{
    string dirName = "C:\\\\";
    // если папка существует
    if (Directory.Exists(dirName))
    {
        Console.WriteLine("Подкаталоги:");
        string[] dirs = Directory.GetDirectories(dirName);
        foreach (string s in dirs)
        {
            Console.WriteLine(s);
        }
        Console.WriteLine();
        Console.WriteLine("Файлы:");
        string[] files = Directory.GetFiles(dirName);
        foreach (string s in files)
        {
            Console.WriteLine(s);
        }
    }
}
```

Консоль отладки Microsoft Visual Studio

C:\\$Recycle.Bin
C:\AHCache
C:\Documents and Settings
C:\eb763b45213b5f2c23484be9
C:\Games
C:\inetpub
C:\OneDriveTemp
C:\PerfLogs
C:\Program Files
C:\Program Files (x86)
C:\ProgramData
C:\Recovery
C:\System Volume Information
C:\Users
C:\Windows
C:\XboxGames

Файлы:

C:\.GamingRoot
C:\DumpStack.log
C:\DumpStack.log.tmp
C:\hiberfil.sys
C:\pagefile.sys
C:\swapfile.sys

C:\Users\nikka\source\repos\Console
(0x0)

Обратите внимание на использование слешей в именах файлов. Либо мы используем двойной слеш: "C:\\", либо одинарный, но тогда перед всем путем ставим знак @: @"C:\Program Files"

Ссылка: 0

```
static void Main(string[] args)
{
    string dirName = @"C:\";

    DirectoryInfo directory = new DirectoryInfo(dirName);

    if (directory.Exists)
    {
        Console.WriteLine("Подкаталоги:");
        DirectoryInfo[] dirs = directory.GetDirectories();
        foreach (DirectoryInfo dir in dirs)
        {
            Console.WriteLine(dir.FullName);
        }
        Console.WriteLine();
        Console.WriteLine("Файлы:");
        FileInfo[] files = directory.GetFiles();
        foreach (FileInfo file in files)
        {
            Console.WriteLine(file.FullName);
        }
    }
}
```

Консоль отладки Microsoft Visual S

```
C:\$Recycle.Bin
C:\AHCache
C:\Documents and Settings
C:\eb763b45213b5f2c23484be9
C:\Games
C:\inetpub
C:\OneDriveTemp
C:\PerfLogs
C:\Program Files
C:\Program Files (x86)
C:\ProgramData
C:\Recovery
C:\System Volume Information
C:\Users
C:\Windows
C:\XboxGames
```

Файлы:

```
C:\.GamingRoot
C:\DumpStack.log
C:\DumpStack.log.tmp
C:\hiberfil.sys
C:\pagefile.sys
C:\swapfile.sys
```

```
C:\Users\nikka\source\repos\Cor
(0x0).
```

Чтобы автоматически закрывать к
томатически закрыть консоль при

Класс FileInfo

Подобно паре **Directory/DirectoryInfo** для работы с файлами предназначена пара классов **File** и **FileInfo**. С их помощью мы можем создавать, удалять, перемещать файлы, получать их свойства и многое другое.

Некоторые полезные методы и свойства класса **FileInfo**:

CopyTo(path): копирует файл в новое место по указанному пути path

Create(): создает файл

Delete(): удаляет файл

MoveTo(destFileName): перемещает файл в новое место

Свойство **Directory**: получает родительский каталог в виде объекта DirectoryInfo

Свойство **DirectoryName**: получает полный путь к родительскому каталогу

Свойство **Exists**: указывает, существует ли файл

Свойство **Length**: получает размер файла

Свойство **Extension**: получает расширение файла

Свойство **Name**: получает имя файла

Свойство **FullName**: получает полное имя файла

Для создания **объекта FileInfo** применяется конструктор, который получает в качестве параметра путь к файлу:

```
FileInfo fileInf = new FileInfo(@"C:\app\content.txt");
```


Класс File

- Класс **File** реализует похожую функциональность с помощью **статических** методов:
- **Copy()**: копирует файл в новое место
- **Create()**: создает файл
- **Delete()**: удаляет файл
- **Move**: перемещает файл в новое место
- **Exists(file)**: определяет, существует ли файл
- Для работы с файлами можно применять как абсолютные,
- так и относительные пути:

```
// абсолютные пути
string path1 = @"C:\Users\eugene\Documents\content.txt"; // для Windows
string path2 = "C:\\Users\\eugene\\Documents\\content.txt"; // для Windows
string path3 = "/Users/eugene/Documents/content.txt"; // для MacOS/Linux

// относительные пути
string path4 = "MyDir\\content.txt"; // для Windows
string path5 = "MyDir/content.txt"; // для MacOS/Linux
```

Получение информации о файле

```
string path = @"C:\Users\nikka\source\repos\ConsoleApp2\ConsoleApp2\Texts\text.txt";  
// создаём объект  
FileInfo fileInfo = new FileInfo(path);  
// если файл существует  
if (fileInfo.Exists)  
{  
    // свойства файла  
    Console.WriteLine($"Имя файла: {fileInfo.Name}");  
    Console.WriteLine($"Время создания: {fileInfo.CreationTime}");  
    Console.WriteLine($"Размер: {fileInfo.Length}");  
}
```

Консоль отладки Microsoft Visual Studio

Имя файла: text.txt
Время создания: 14.01.2026 5:34:08
Размер: 14

Удаление файла

```
string path = @"C:\Users\nikka\source\repos\ConsoleApp2\ConsoleApp2\Texts\text.txt";  
// создаём объект  
FileInfo fileInf = new FileInfo(path);  
// проверяем существует ли файл  
if (fileInf.Exists)  
{  
    fileInf.Delete();  
    Console.WriteLine(fileInf.Name);  
    // альтернатива с помощью класса File  
    // File.Delete(path);  
}  
fileInf = new FileInfo(path);  
if (!fileInf.Exists)  
{  
    Console.WriteLine("Файл не найден");  
}
```

Консоль отладки Microsoft Visual Studio

text.txt
Файл не найден

C:\Users\nikka\source\repos\ConsoleApp2\ConsoleA
(0x0).
Чтобы автоматически закрывать консоль при остано
томатически закрыть консоль при остановке отладк
Нажмите любую клавишу, чтобы закрыть это окно:

Чтение и запись файлов

В дополнение к вышерассмотренным методам класс `File` также предоставляет ряд методов для чтения-записи текстовых и бинарных файлов:

- **`AppendAllLines(String, IEnumerable<String>) / AppendAllLinesAsync(String, IEnumerable<String>, CancellationToken)`**

добавляют в файл набор строк. Если файл не существует, то он создается

- **`AppendAllText(String, String) / AppendAllTextAsync(String, String, CancellationToken)`**

добавляют в файл строку. Если файл не существует, то он создается

- **`byte[] ReadAllBytes (string path) / Task<byte[]> ReadAllBytesAsync (string path, CancellationToken cancellationToken)`**

считывают содержимое бинарного файла в массив байтов

- **`string[] ReadAllLines (string path) / Task<string[]> ReadAllLinesAsync (string path, CancellationToken cancellationToken)`**

считывают содержимое текстового файла в массив строк

- **`string ReadAllText (string path) / Task<string> ReadAllTextAsync (string path, CancellationToken cancellationToken)`**

считывают содержимое текстового файла в строку

- **`IEnumerable<string> ReadLines (string path)`**

считывают содержимое текстового файла в коллекцию строк

Чтение и запись файлов

- **void WriteAllBytes (string path, byte[] bytes) / Task WriteAllBytesAsync (string path, byte[] bytes, CancellationToken cancellationToken)**

записывают массив байт в бинарный файл. Если файл не существует, он создается. Если существует, то перезаписывается

- **void WriteAllLines (string path, string[] contents) / Task WriteAllLinesAsync (string path, IEnumerable<string> contents, CancellationToken cancellationToken)**

записывают массив строк в текстовый файл. Если файл не существует, он создается. Если существует, то перезаписывается

- **WriteAllText (string path, string? contents) / Task WriteAllTextAsync (string path, string? contents, CancellationToken cancellationToken)**

записывают строку в текстовый файл. Если файл не существует, он создается. Если существует, то перезаписывается

```
string path = @"C:\Users\nikka\source\repos\ConsoleApp2\ConsoleApp2\Texts\text.txt";
```

```
string originalText = "Hello Metanit.com";  
// запись строки  
File.WriteAllText(path, originalText);  
// дозапись в конец файла  
File.AppendAllText(path, "\nHello work");  
  
// чтение файла  
string fileText = File.ReadAllText(path);  
Console.WriteLine(fileText);
```

Консоль отладки Microsoft Visual Studio

```
Hello Metanit.com  
Hello work
```

```
C:\Users\nikka\source\repos\ConsoleApp2\ConsoleApp2\bin\Deb  
(0x0).
```

Чтобы автоматически закрывать консоль при остановке отладки
томатически закрыть консоль при остановке отладки".

Нажмите любую клавишу, чтобы закрыть это окно:

text.txt x Program.cs

```
1 Hello Metanit.com  
2 Hello work
```

Стоит отметить, что при добавлении текста я добавил в строку последовательность "\n", которая выполняет перевод на следующую строку. Благодаря этому добавляемый текст располагается в файле на новой строке.

Если мы хотим, что в файле изначально шло добавление на новую строку, то для записи стоит использовать метод WriteAllLines / WriteAllLinesAsync, а для добавления - AppendAllLines / AppendAllLinesAsync

```
File.WriteAllLines(path, new[] { "Hello Metanit.com", "Hello work" });
```

Аналогично при чтении файла если мы хотим каждую строку файла считать отдельно, то вместо ReadAllText / ReadAllTextAsync применяется ReadAllLines / ReadAllLinesAsync.

Кодировка

В качестве дополнительного параметра методы чтения-записи текстовых файлов позволяют установить кодировку в виде объекта `System.Text.Encoding`.

Для установки кодировки при записи и чтении здесь применяется встроенное значение `Encoding.Unicode`. Также можно указать название кодировки, единственное следует удостовериться, что текущая операционная система поддерживает выбранную кодировку

```
string path = "C:\\Users\\nikka\\source\\repos\\ConsoleApp2\\ConsoleApp2\\Texts\\text.txt";

string originalText = "Привет Metanit.com";
// запись строки
File.WriteAllText(path, originalText, Encoding.Unicode);
// дозапись в конец файла
File.AppendAllText(path, "\nПривет мир", Encoding.Unicode);

// чтение файла
string fileText = File.ReadAllText(path, Encoding.Unicode);
Console.WriteLine(fileText);
```

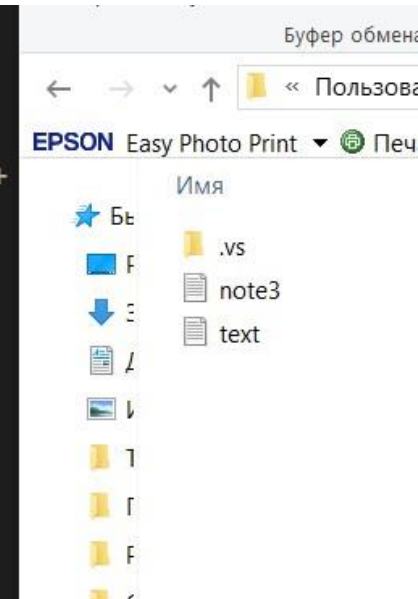
FileStream. Чтение и запись файла

- Класс `FileStream` представляет возможности по считыванию из файла и записи в файл. Он позволяет работать как с текстовыми файлами, так и с бинарными.
- Создание `FileStream`
- Для создания объекта `FileStream` можно использовать как конструкторы этого класса, так и статические методы класса `File`. Конструктор `FileStream` имеет множество перегруженных версий, из которых отмечу лишь одну,
- самую простую и используемую:
- *`FileStream(string filename, FileMode mode)`*
- Здесь в конструктор передается два параметра: путь к файлу и перечисление `FileMode`. Данное перечисление указывает на режим доступа к файлу и может принимать следующие значения:
 - **Append**: если файл существует, то текст добавляется в конец файл. Если файла нет, то он создается. Файл открывается только для записи.
 - **Create**: создается новый файл. Если такой файл уже существует, то он перезаписывается
 - **CreateNew**: создается новый файл. Если такой файл уже существует, то приложение выбрасывает ошибку
 - **Open**: открывает файл. Если файл не существует, выбрасывается исключение
 - **OpenOrCreate**: если файл существует, он открывается, если нет - создается новый
 - **Truncate**: если файл существует, то он перезаписывается. Файл открывается только для записи.

Заккрытие потока

Класс `FileStream` для освобождения всех ресурсов, связанных с файлом, реализует интерфейс `IDisposable`. Соответственно после завершения работы с `FileStream` необходимо освободить связанный с ним файл вызовом метода `Dispose`. Для корректного закрытия можно вызвать метод `Close()`, который вызывает метод `Dispose`:

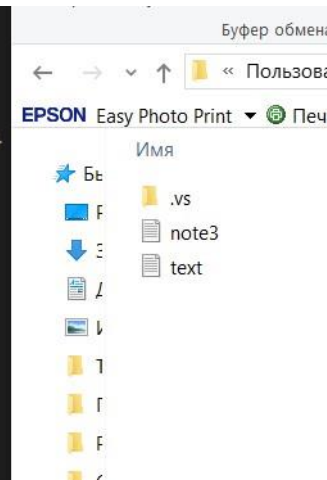
```
FileStream fstream = null;
try
{
    fstream = new FileStream("C:\\Users\\nikka\\source\\repos\\ConsoleApp2" +
        "\\ConsoleApp2\\Texts\\note3.txt", FileMode.OpenOrCreate);
    // операции с fstream
}
catch (Exception ex)
{ }
finally
{
    fstream?.Close();
}
```



Заккрытие потока

Класс FileStream для освобождения всех ресурсов, связанных с файлом, реализует интерфейс IDisposable. Соответственно после завершения работы с FileStream необходимо освободить связанный с ним файл вызовом метода Dispose. Для корректного закрытия можно вызвать метод Close(), который вызывает метод Dispose:

```
FileStream fstream = null;
try
{
    fstream = new FileStream("C:\\Users\\nikka\\source\\repos\\ConsoleApp2" +
        "\\ConsoleApp2\\Texts\\note3.txt", FileMode.OpenOrCreate);
    // операции с fstream
}
catch (Exception ex)
{ }
finally
{
    fstream?.Close();
}
```



Либо можно использовать конструкцию `using`, которая автоматически освободит все связанные с `FileStream` ресурсы:

```
using (FileStream fstream = new FileStream("note3.txt", FileMode.OpenOrCreate))
{
    // операции с fstream
}
```

Свойства и методы FileStream

Рассмотрим наиболее важные свойства класса **FileStream**:

- Свойство **Length**: возвращает длину потока в байтах
- Свойство **Position**: возвращает текущую позицию в потоке
- Свойство **Name**: возвращает абсолютный путь к файлу, открытому в FileStream

Для чтения/записи файлов можно применять следующие методы класса **FileStream**:

- `void CopyTo(Stream destination)`: копирует данные из текущего потока в поток destination
- `Task CopyToAsync(Stream destination)`: асинхронная версия метода CopyTo
- `void Flush()`: сбрасывает содержимое буфера в файл
- `Task FlushAsync()`: асинхронная версия метода Flush

- `int Read(byte[] array, int offset, int count)`: считывает данные из файла в массив байтов и возвращает количество успешно считанных байтов. Принимает три параметра:
 - `array` - массив байтов, куда будут помещены считываемые из файла данные
 - `offset` представляет смещение в байтах в массиве `array`, в который считанные байты будут помещены
 - `count` - максимальное число байтов, предназначенных для чтения. Если в файле находится меньшее количество байтов, то все они будут считаны.
 - `Task<int> ReadAsync(byte[] array, int offset, int count)`: асинхронная версия метода `Read`
- `long Seek(long offset, SeekOrigin origin)`: устанавливает позицию в потоке со смещением на количество байт, указанных в параметре `offset`.
- `void Write(byte[] array, int offset, int count)`: записывает в файл данные из массива байтов. Принимает три параметра:
 - `array` - массив байтов, откуда данные будут записываться в файл
 - `offset` - смещение в байтах в массиве `array`, откуда начинается запись байтов в поток
 - `count` - максимальное число байтов, предназначенных для записи
- `Task WriteAsync(byte[] array, int offset, int count)`: асинхронная версия метода `Write`

Чтение и запись файлов

FileStream представляет доступ к файлам на уровне байтов, поэтому, например, если вам надо считать или записать одну или несколько строк в текстовый файл, то массив байтов надо преобразовать в строки, используя специальные методы. Поэтому для работы с текстовыми файлами применяются другие классы.

В то же время при работе с различными бинарными файлами, имеющими определенную структуру, FileStream может быть очень даже полезен для извлечения определенных порций информации и ее обработки.

Алгоритм записи и чтения файла:

1. Определяем путь файла и создаем FileStream с этим путем, желательно использовать using
2. Для записи кодируем текст в массив байт
3. Записываем массив байт в файл
4. Для чтения создаем пустой массив под байты которые будем читать из файла
5. Читаем файл, указывая массив для заполнения
6. Декодируем байты в строку

```

// данный файл будет лежать в bin/Debug
string path = "note.txt"; // путь к файлу

string text = "мега крутой текст"; // строка для записи

// запись в файл
using (FileStream fstream = new FileStream(path, FileMode.OpenOrCreate))
{
    // преобразуем строку в байты
    byte[] buffer = Encoding.Default.GetBytes(text);
    // запись массива байтов в файл
    fstream.Write(buffer, 0, buffer.Length);
    Console.WriteLine("Текст записан в файл");
}

// чтение из файла
using (FileStream fstream = File.OpenRead(path))
{
    // выделяем массив для считывания данных из файла
    byte[] buffer = new byte[fstream.Length];
    // считываем данные
    fstream.Read(buffer, 0, buffer.Length);
    // декодируем байты в строку
    string textFromFile = Encoding.Default.GetString(buffer);
    Console.WriteLine($"Текст из файла: {textFromFile}");
}

```

Разберем этот пример. Вначале определяем путь к файлу и текст для записи в файл.

И при чтении, и при записи для создания и удаления объекта `FileStream` используется конструкция `using`, по завершению которой у созданного объекта `FileStream` автоматически вызывается метод `Dispose`, и, таким образом, объект уничтожается.

И при записи, и при чтении применяется объект кодировки `Encoding.Default` из пространства имен `System.Text`. В данном случае мы используем два его метода: `GetBytes` для получения массива байтов из строки и `GetString` для получения строки из массива байтов.

Произвольный доступ к файлам

Нередко бинарные файлы представляют определенную структуру. И, зная эту структуру, мы можем взять из файла нужную порцию информации или наоборот записать в определенном месте файла определенный набор байтов. Например, в wav-файлах непосредственно звуковые данные начинаются с 44 байта, а до 44 байта идут различные метаданные - количество каналов аудио, частота дискретизации и т.д.

С помощью метода **Seek()** мы можем управлять положением курсора потока, начиная с которого производится считывание или запись в файл. Этот метод принимает два параметра: **offset** (смещение) и позиция в файле. Позиция в файле описывается тремя значениями:

SeekOrigin.Begin: начало файла

SeekOrigin.End: конец файла

SeekOrigin.Current: текущая позиция в файле

Курсор потока, с которого начинается чтение или запись, **смещается вперед** на значение **offset** относительно позиции, указанной в качестве второго параметра. Смещение может быть отрицательным, тогда курсор сдвигается назад, если положительное - то вперед.

Чтение с определённой позиции

```
string path = "note.txt";

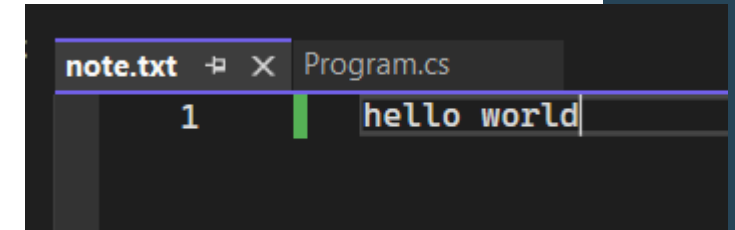
string text = "hello world";

using (FileStream fstream = new FileStream(path, FileMode.OpenOrCreate))
{
    // преобразуем строку в байты
    byte[] input = Encoding.Default.GetBytes(text);
    // запись массива байтов в файл
    fstream.Write(input, 0, input.Length);
    Console.WriteLine("Текст записан в файл");
}

// чтение части файла
using (FileStream fstream = new FileStream(path, FileMode.Open))
{
    // перемещаем указатель в конец файла, до конца файла – пять байт
    fstream.Seek(-5, SeekOrigin.End); // минус 5 символов с конца потока

    // считываем четыре символов с текущей позиции
    byte[] output = new byte[5];
    fstream.Read(output, 0, output.Length);
    // декодируем байты в строку
    string textFromFile = Encoding.Default.GetString(output);
    Console.WriteLine($"Текст из файла: {textFromFile}"); // world
}
```

Текст записан в файл
Текст из файла: world



Алгоритм чтения:

1. Перемещаем курсор, на тот символ с которого хотим прочитать файл
2. Создаем массив с количеством символов которых хотим прочитать
3. Читаем файл и записываем в массив
4. Декодируем байты

Вначале записываем в файл текст "hello world". Затем снова обращаемся к файлу для считывания. Сначала перемещаем курсор на пять символов назад относительно конца файлового потока.

То есть после выполнения этого вызова курсор будет стоять на позиции символа "w".

После этого считываем пять байт начиная с символа "w". В кодировке по умолчанию 1 символ будет представлять 1 байт. Поэтому чтение 5 байт будет эквивалентно чтению пяти символов: "world".



Перезапись с определённой позиции

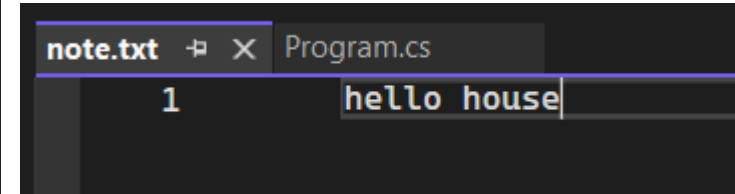
```
string path = "note.txt";

string text = "hello world";

// запись в файл
using (FileStream fstream = new FileStream(path, FileMode.OpenOrCreate))
{
    // преобразуем строку в байты
    byte[] input = Encoding.Default.GetBytes(text);
    // запись массива байтов в файл
    fstream.Write(input, 0, input.Length);
    Console.WriteLine("Текст записан в файл");
}

using (FileStream fstream = new FileStream(path, FileMode.OpenOrCreate))
{
    // заменим в файле слово world на слово house
    string replaceText = "house";
    fstream.Seek(-5, SeekOrigin.End); // минус 5 символов с конца потока
    byte[] input = Encoding.Default.GetBytes(replaceText);
    // указатель находится на 6м символе с начала то есть "hello |world"
    // поэтому запись происходит с 6го символа, до 6 + 5,
    // то есть символы слова "world" заменяются на символы "house"
    fstream.Write(input, 0, input.Length);

    // считываем весь файл
    // возвращаем указатель в начало файла
    fstream.Seek(0, SeekOrigin.Begin);
    byte[] output = new byte[fstream.Length];
    // чтение всей строки т.к. указатель в начале
    fstream.Read(output, 0, output.Length);
    // декодируем байты в строку
    string textFromFile = Encoding.Default.GetString(output);
    Console.WriteLine($"Текст из файла: {textFromFile}"); // hello house
}
```



```
Текст записан в файл
Текст из файла: hello house
```

Здесь также вначале записываем в файл строку "hello world". Затем также открываем файл и опять же перемещаемся в конец файла, не доходя до конца пять символов (то есть опять же с позиции символа "w"), и осуществляем запись строки "house". Таким образом, строка "house" заменяет строку "world".

Дозапись с определённой позиции

```
using (FileStream fstream = new FileStream(path, FileMode.OpenOrCreate))
{
    // позиция перед "world" (после пробела, позиция 6)
    int insertPos = 6;
    fstream.Seek(insertPos, SeekOrigin.Begin);

    // читаем оставшуюся часть после позиции вставки ("world")
    byte[] tail = new byte[fstream.Length - insertPos];
    fstream.Read(tail, 0, tail.Length);

    // вставляем "house " (пробел для разделения)
    string insertText = "house ";
    byte[] insertBytes = Encoding.Default.GetBytes(insertText);
    fstream.Seek(insertPos, SeekOrigin.Begin);
    fstream.Write(insertBytes, 0, insertBytes.Length);

    // перезаписываем сдвинутый tail в конец
    fstream.Seek(insertPos + insertBytes.Length, SeekOrigin.Begin);
    fstream.Write(tail, 0, tail.Length);

    // считываем весь файл
    // возвращаем указатель в начало файла
    fstream.Seek(0, SeekOrigin.Begin);
    byte[] output = new byte[fstream.Length];
    fstream.Read(output, 0, output.Length);
    // декодируем байты в строку
    string textFromFile = Encoding.Default.GetString(output);
    Console.WriteLine($"Текст из файла: {textFromFile}"); // hello house world
}
```

Текст записан в файл
Текст из файла: hello house world world

Чтобы вставить слово не в конце, а посреди текста, нужно сначала записать это слово, затем дописать слова которые начинались с той позиции, куда было вставлено слово.

Задание: Консольное приложение для работы с FileStream

Цель: Разработать консольное приложение с меню, демонстрирующее основные возможности класса FileStream для работы с файлами.

Создать для каждого пункта **методы**:

1. Создать файл (FileMode.Create) - создает новый пустой файл с именем data.txt
2. Заполнить файл через консоль (FileMode.Truncate) -
 - Запрашивает текст у пользователя
 - Полностью перезаписывает содержимое файла введенным текстом
3. Удалить файл (File.Delete) - удаляет файл data.txt если он существует
4. Добавить строку в конец (FileMode.Append) -
 - Запрашивает строку у пользователя
 - Добавляет её в конец файла с переводом курсора в начало \r и переводом строки \n (\r\n)
5. Очистить файл (FileMode.Truncate) - устанавливает размер файла в 0 байт

6. Чтение с конкретного символа -

- Запрашивает номер символа (позицию)
- Читает все данные от указанной позиции до конца файла
- Выводит результат на консоль

7. Дозапись с конкретного символа (FileAccess.ReadWrite) -

- Запрашивает номер символа и текст для записи
- Перезаписывает данные с указанной позиции
- Если позиция больше размера файла - добавляет в конец

Требования:

- Меню в цикле с возможностью выхода (пункт 0)
- Проверки существования файла перед операциями